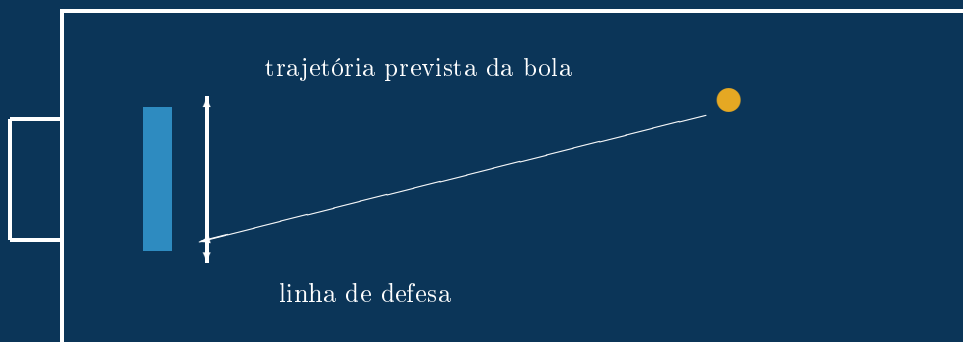


Treinamento de um Goleiro

para Futebol de Robôs VSSS

Campos vetoriais, controle e computação evolutiva



Documento introdutório para estudantes de graduação

VSSS Coach – setembro de 2026

Visão geral

O objetivo é treinar um robô para atuar como goleiro em uma partida de futebol VSSS. Ele deve permanecer diante do próprio gol, prever onde a bola cruzará sua linha de defesa e chegar a esse ponto a tempo. O treinamento não ensina cada comando de roda diretamente. Em vez disso, procura bons valores para os parâmetros de uma estratégia já estruturada.

Ideia central. Um campo vetorial sugere para onde o robô deve se mover. Um controlador transforma essa sugestão em velocidades das rodas. A computação evolutiva testa várias configurações, conserva as melhores e gera novas tentativas.

O processo usa o Webots/TraveSim para simular a física. Cada candidato enfrenta o mesmo conjunto de chutes: posições variadas da bola e do goleiro, diferentes direções e cinco velocidades iniciais. Assim, uma configuração não ganha apenas por ter recebido chutes mais fáceis.

Antes de continuar: o ambiente VSSS

VSSS é uma modalidade de futebol com robôs pequenos em um campo reduzido. Uma câmera ou o simulador fornece as posições e velocidades. O robô usado aqui é **diferencial**: possui duas rodas motorizadas independentes. Se as rodas giram juntas, ele anda; se giram em sentidos opostos, ele muda sua orientação. Neste experimento isolamos um goleiro e repetimos chutes controlados para estudar seu movimento antes de colocá-lo em uma partida completa.

1 O que é um campo vetorial?

Imagine pequenas setas desenhadas em muitos pontos do campo. Em cada posição $\mathbf{p} = (x, y)$, uma função fornece uma seta

$$\mathbf{F}(\mathbf{p}) = (F_x(\mathbf{p}), F_y(\mathbf{p})).$$

A direção da seta indica o sentido desejado do movimento; seu comprimento pode representar a intensidade. O robô não precisa planejar uma trajetória completa de uma só vez: ele observa o estado atual, consulta o campo, move-se um pouco e repete o cálculo.

O termo **campo univetorial** designa uma família de campos que combina aproximação ao alvo com orientação final. Perto da bola, as setas podem curvar a trajetória para que o robô chegue pelo lado apropriado, em vez de simplesmente apontar para o centro da bola. O raio controla o tamanho dessa curva; a suavidade controla quão gradual é a transição. No goleiro, esse campo auxilia a aproximação, mas a interceptação preditiva determina o alvo quando há ameaça.

No goleiro, o campo mais importante é o de **interceptação preditiva**. Em vez de apontar para a posição atual da bola, ele aponta para a posição na qual a bola provavelmente cruzará uma linha diante do gol. Com posição $\mathbf{b}$, velocidade $\mathbf{v}$ e aceleração $\mathbf{a}$ da bola, a previsão simples é

$$\mathbf{b}(t) = \mathbf{b}_0 + \mathbf{v}_0 t + \frac{1}{2} \mathbf{a} t^2.$$

O instante positivo em que a coordenada horizontal atinge a linha do goleiro é estimado. A coordenada vertical prevista é limitada ao espaço útil entre as traves, com uma margem de segurança. O vetor desejado é então

$$\mathbf{F}_{\text{int}} = \frac{\mathbf{p}_a - \mathbf{p}_r}{\|\mathbf{p}_a - \mathbf{p}_r\|},$$

em que os índices a e r representam alvo e robô.

Essa fórmula fornece apenas a **direção**. A velocidade é calculada à parte. Em forma compacta,

$$\mathbf{v}_{\text{desejada}} = s(d, \mathbf{v}_b, \mathbf{a}_b, t) \mathbf{F}_{\text{int}},$$

em que s cresce quando o alvo está distante ou urgente e diminui perto da bola para melhorar o controle. Portanto, uma seta unitária não significa velocidade constante.

Exemplo simples. A linha está em $x = -0,69$ m. A bola está em $(0,21; 0,10)$ m e viaja para a esquerda a $0,90$ m/s, sem aceleração lateral. Ela levará aproximadamente $(0,21 + 0,69)/0,90 = 1$ s para chegar à linha. Com velocidade vertical de $-0,05$ m/s, o alvo previsto fica perto de $y = 0,05$ m. O goleiro começa a se deslocar agora, sem esperar a bola chegar.

Se a bola não estiver indo em direção ao gol, o goleiro continua próximo de sua linha, em vez de persegui-la pelo campo.

Campos, controle e prioridades

Nem todo comportamento é um campo evoluído. A estratégia é organizada em camadas, nesta ordem conceitual:

Camada	Função	Quando atua
Segurança e limites	Impede comandos inválidos e limita as rodas.	Sempre; maior prioridade.
Rebatida por giro	Mantém contato e gira para afastar a bola.	Após tocar a bola.
Interceptação preditiva	Atraí para o cruzamento previsto na linha.	Bola indo para o gol.
Retorno e alinhamento	Recupera a linha e orienta o chassi.	Fora da linha ou desalinhado.
Posição neutra	Mantém o goleiro diante da boca do gol.	Sem ameaça válida.

O corredor defensivo também possui ganho ajustável, mas é usado principalmente no defensor. Para o goleiro predominam interceptação, retorno e rebatida.

Na prática, essas camadas formam uma pequena **state machine** (máquina de estados). Durante uma ameaça, a interceptação escolhe o alvo, enquanto o controlador ainda corrige a orientação. Após o contato, a rebatida suspende temporariamente o alinhamento; quando termina, o retorno recupera a linha. Segurança e limites vencem todos os demais comandos.

2 Como o goleiro permanece paralelo ao gol

A linha de defesa é vertical no referencial do simulador e fica alguns centímetros à frente da linha do gol. A orientação desejada do chassi é 90 graus: assim, as rodas ficam alinhadas para o robô avançar ou recuar ao longo dessa linha sem precisar girar a cada mudança de direção.

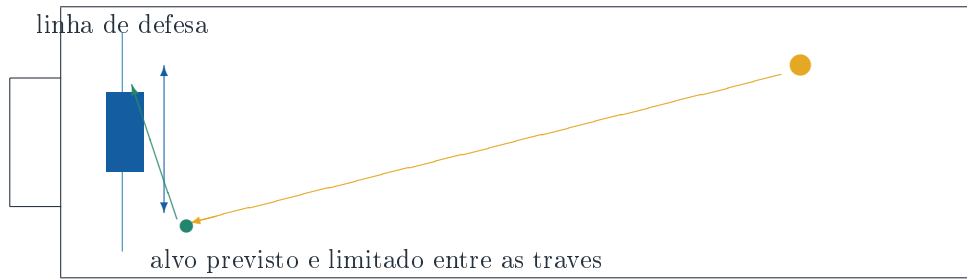


Figura 1 – Referencial adotado: o eixo horizontal x atravessa o campo e o eixo vertical y acompanha a boca do gol. A linha azul é a linha de defesa; o ponto verde é o cruzamento previsto da bola.

O controle ocorre em duas etapas:

1. Se o robô estiver fora da linha por mais de 1,5 cm, ele primeiro corrige esse erro transversal.
2. Na linha, ele escolhe avançar ou recuar pelo caminho que exige a menor correção angular. Se estiver desalinhado por mais de 5 graus, para a translação e corrige a orientação antes de continuar. Para recuar, mantém a orientação e apenas inverte o sinal da velocidade das duas rodas.

Para um robô diferencial, velocidade linear v e angular ω viram velocidades das rodas:

$$\omega_L = \frac{v - \frac{d}{2}\omega}{r}, \quad \omega_R = \frac{v + \frac{d}{2}\omega}{r},$$

onde r é o raio da roda e d é a distância entre rodas. Os comandos são limitados pela capacidade física. Após contato com a bola, o goleiro espera um pequeno tempo ajustável, mantém contato e executa um giro de 180° para afastá-la. Esse giro é uma exceção curta à regra de permanecer paralelo. Ao terminar, o controlador volta ao estado de alinhamento. Uma histerese impede que o giro seja disparado repetidamente.

3 Como são criados os testes

Cada cenário define posição inicial da bola, posição do goleiro, direção do chute, velocidade inicial e duração. Os chutes miram diferentes pontos da boca do gol e usam velocidades entre 0,75 e 2,00 m/s. O goleiro começa em posições variadas ao longo de sua linha.

Cenários impossíveis são rejeitados durante o treinamento. Uma verificação conservadora considera tempo de reação de 150 ms, aceleração máxima de $2,5 \text{ m/s}^2$, velocidade máxima de 1,7 m/s e margem física de contato. Isso evita punir um candidato por uma defesa que nenhum robô poderia realizar. Depois do aprendizado, cenários mais difíceis podem ser usados para avaliar robustez.

A velocidade de até 2,5 m/s citada adiante é uma referência desejada antes da saturação. A velocidade física alcançável, usada no teste de viabilidade, é 1,7 m/s. O controlador pode pedir mais, mas os limites do robô reduzem o comando.

Todos os candidatos de uma geração recebem exatamente os mesmos cenários, definidos por uma **seed** (semente aleatória) registrada. Essa comparação justa é chamada, em estatística computacional, de *common random numbers* (CRN).

Na execução atual, esse conjunto permanece fixo durante todas as gerações de uma mesma execução. Assim, a melhora não pode ser explicada pela troca dos chutes. Ao final, recomenda-se uma validação com outra *seed*, nunca vista no treinamento, para verificar se o goleiro aprendeu uma regra geral em vez de decorar os casos.

4 Loss (função de perda)

Uma **loss** é um número que queremos minimizar. Zero é melhor. Para cada chute são calculados quatro componentes entre 0 e 1:

Componente	Interpretação	Peso
Resultado	1 se o goleiro sofre gol; 0 caso contrário.	70%
Interceptação	Menor distância entre robô e bola, dividida por 0,75 m e limitada a $[0, 1]$.	12%
Reação	Tempo até o contato dividido pela duração do cenário.	8%
Redirecionamento	0 se o contato muda a trajetória da bola; 1 se não muda.	10%

$$L = 0,70L_{\text{res}} + 0,12L_{\text{int}} + 0,08L_{\text{rea}} + 0,10L_{\text{red}}$$

O peso maior está no que realmente importa: não sofrer gol. Os outros termos da *loss* dão informação mesmo quando dois candidatos obtêm o mesmo resultado. Por exemplo, entre dois goleiros que defenderam, favorece-se aquele que chegou mais perto, reagiu antes e afastou a bola.

Exemplo numérico. Suponha que não houve gol ($L_{\text{res}} = 0$), que a distância normalizada foi 0,20, o tempo de reação foi 0,30 e a bola foi redirecionada ($L_{\text{red}} = 0$). Então $L = 0,12(0,20) + 0,08(0,30) = 0,048$. Se não houver contato, reação e redirecionamento recebem o pior valor, 1. Isso evita premiar um robô que apenas teve sorte porque o chute saiu para fora.

Cuidado ao interpretar zero. $L_{\text{res}} = 0$ significa que nenhum gol foi sofrido nos cenários avaliados. Isso não prova que todas as bolas foram tocadas, pois um chute pode errar o gol. Já a *loss* total dificilmente é zero, porque distância e tempo de reação continuam positivos. O valor usado por uma geração é a média dos valores de *loss* de todos os seus cenários. Por isso, a análise deve separar gols sofridos, contatos ou defesas e chutes que erraram o alvo.

5 O que é otimizado

O algoritmo não inventa todo o controlador. Ele ajusta parâmetros contínuos, dentro de limites seguros. A tabela resume os principais grupos; não é uma lista exaustiva de cada variável interna:

Grupo	O que é ajustado	Efeito visível
Campo univetorial	Raio de 0,04–0,30 m e suavidade de 0,01–0,30.	Curva maior ou chegada mais direta à bola.
Perfil de velocidade	Velocidades perto/longe (0,15–2,50 m/s) e transições (0,03–1,40 m).	Acelera distante e controla o contato perto.
Previsão	Horizonte de 0,20–2,50 s e linha a 0,045–0,090 m do gol.	Antecipa mais cedo e fica mais perto ou longe do gol.
Interceptação	Margem das traves, ganho, mistura e velocidade mínima da ameaça.	Decide quão forte e quando perseguir o cruzamento.
Antecipação	Ganhos de velocidade, aceleração e urgência; folga até 0,35 s.	Reage mais cedo a chutes rápidos.
Rebatida	Espera de 0,02–0,30 s antes do giro.	Mantém contato antes de afastar a bola.

A **mistura** combina duas sugestões:

$$\mathbf{F} = \alpha \mathbf{F}_{\text{int}} + (1 - \alpha) \mathbf{F}_{\text{base}}, \quad 0 \leq \alpha \leq 1.$$

Com α próximo de 1, domina a interceptação prevista; próximo de 0, domina a aproximação univetorial básica. A velocidade mínima da ameaça decide a partir de qual movimento da bola essa interceptação passa a ser considerada.

O campo de corredor defensivo também possui ganho ajustável, embora seja mais importante para o defensor. Limites evitam soluções perigosas ou incompatíveis com a física. Regras de segurança, saturação das rodas e validação numérica não são evoluídas.

6 Computação evolutiva, sem mistério

Computação evolutiva procura soluções por população. Cada candidato é um vetor com todos os parâmetros acima. Pense em cada vetor como uma receita: os ingredientes são as mesmas equações e as quantidades são os parâmetros. A cada geração, várias receitas são testadas nos mesmos chutes. O ciclo é:

1. gerar candidatos → 2. simular os mesmos chutes → 3. medir a loss
→ 4. selecionar os melhores → 5. recombinar e variar → nova geração

No método utilizado, o melhor quarto da população recebe pesos maiores. A média ponderada desses candidatos vira o centro da próxima busca. Cada parâmetro tem seu próprio desvio padrão: se os bons candidatos ainda são variados naquele parâmetro, a busca continua ampla; se concordam, ela se concentra. Um piso de 1% da faixa mantém exploração.

Há também um **surrogate model** (modelo substituto): uma regressão linear com regularização *Ridge* aprende localmente a relação entre parâmetros testados e valores de *loss*. Ela funciona como um mapa aproximado da região já explorada e sugere uma pequena mudança da média para reduzir a *loss*. É um atalho para gastar menos simulações, não uma substituição da física.

Como exemplo de escala, uma execução com 32 candidatos, 30 chutes e 16 gerações avalia $32 \times 30 \times 16 = 15\,360$ cenários. É por isso que executar cenários em paralelo e aproveitar o mapa aproximado pode economizar bastante tempo.

O método é melhor descrito como **estratégia evolutiva diagonal adaptativa assistida por surrogate model**. Não é aprendizado por reforço e não há retropropagação através do Webots. Também não é programação genética em sentido estrito, pois são evoluídos números de equações fixas, e não árvores de expressões.

7 Como acompanhar e interpretar

O *dashboard* mostra a *loss* total por geração, cada componente separadamente, o *ranking* e o *replay* do melhor candidato. O esperado é uma tendência de queda, mas pequenas oscilações são normais: a população continua explorando e a física tem variações. É importante observar simultaneamente:



Figura 2 – Exemplo real de acompanhamento. No painel esquerdo, a *loss* total cai bastante. No painel direito, a *outcome loss* chega perto de zero, enquanto a reação permanece como o componente mais alto; esse é um sinal de que ainda há espaço para antecipar o contato.

- a *loss* de resultado, para saber quantos gols passaram;
- distância e reação, para detectar progresso antes de uma defesa perfeita;
- redirecionamento, para evitar que a bola permaneça perigosa diante do gol;
- vídeos com os mesmos cenários, evitando comparações visuais injustas.

Os vídeos “antes e depois” usam as trajetórias originais gravadas no treino. Quando cada tentativa recebe a mesma duração, uma defesa rápida não encurta um lado do vídeo. Uma nova simulação deve ser identificada como reavaliação, pois ela pode produzir resultado ligeiramente diferente.

Resumo em cinco ideias

1. O campo vetorial transforma o estado da partida em uma direção desejada.
2. A posição futura da bola define um alvo sobre uma linha diante do gol.
3. Um controle diferencial mantém o chassi paralelo e move o robô para frente ou para trás nessa linha.
4. A *loss* combina não sofrer gol com chegar perto, reagir cedo e afastar a bola.
5. A evolução ajusta parâmetros; o *surrogate model* ajuda a escolher regiões promissoras, sempre confirmadas pela simulação física.

Referências para continuar

- Y. Lim et al. *Evolutionary Univector Field-based Navigation with Collision Avoidance for Mobile Robot*. IFAC World Congress, 2008.
- S. Russell e P. Norvig. *Artificial Intelligence: A Modern Approach*, 4ª edição. Pearson, 2021.
- Futebol Mini. *TraveSim: Webots Simulation Environment for IEEE Very Small Size Soccer*. <https://github.com/futebol-mini/travesim>.

Documento conceitual do projeto VSSS Coach. Os valores apresentados refletem a configuração atual do treinamento e podem evoluir com novos experimentos.